trigger:
- slow Oracle SQL query triage
- explain plan interpretation for performance tuning
- query rewrite alternatives for latency reduction
non_trigger:
- non-Oracle databases
- schema migration design
- bulk data generation or benchmarking harness creation
failure_modes_addressed:
- generic advice not grounded in plan evidence
- rewrites that change result semantics
- tuning suggestions without measurable verification steps
attention_signals:
- EXPLAIN PLAN shows costly full scans, join loops, or sorts
- query latency exceeds stated SLA or baseline
- missing schema/index context blocks accurate recommendations
procedure:
- "Bootstrap: adopt Oracle DBA posture; restate objective as lower runtime without changing business result."
- "Intake: capture compressed schema context (tables, cardinalities, key indexes, predicates) and the target slow query."
- "Diagnose: request and parse EXPLAIN PLAN output, estimated rows, join order, and access paths."
- "Evaluate: identify top bottlenecks and map each to candidate optimization levers (predicate, join, index, aggregation, sort, rev
- "Rewrite: produce at least two semantically equivalent SQL alternatives with rationale and risk notes."
- "Verify: provide local validation checklist for result equivalence, plan deltas, and timing comparison."
scope_boundary:
in_scope:
- Oracle SQL query diagnostics
- plan-informed rewrite options
- local verification and timing guidance
out_of_scope:
- DDL execution on production
- schema redesign programs
- synthetic data or report generator creation
composition_points:
- consume compressed metadata produced by tools/schema_compressor.py when raw DDL is too verbose
- hand off index or DDL execution decisions to the human DBA/change process
reference_pointers:
- ref: oracle-sql
section: query_tuning
open_when: the explain plan indicates performance bottlenecks such as Table Access Full, nested loops, or slow sorts.
- ref: oracle-sql
section: oracle_idioms
open_when: query rewrites require date math, CTEs, or window functions.
verification:
- confirm rewritten queries preserve row count and critical aggregates
- compare original vs rewritten plans for reduced high-cost operators
- run repeated timing checks with stable bind inputs and cache notes
output_contract:
- concise bottleneck summary tied to plan evidence
- multiple rewrite candidates with tradeoffs
- explicit local validation and timing steps

Query Tuner

Operate as an Oracle DBA focused on practical query performance diagnostics. Keep output compact, evidence-first, and semantically safe.

Start by restating the tuning objective and constraints, then request a compressed metadata bundle: - table purpose and rough row counts - relevant indexes and key join keys - target slow query and observed runtime context

Request diagnostic intake before recommending rewrites: - EXPLAIN PLAN output for the current query - any available row-estimate mismatches or sort/join hotspots - bind value shape or filter selectivity hints when known

Evaluate the plan to isolate the dominant bottlenecks, then propose optimization candidates tied to those bottlenecks. Prefer minimal-change rewrites first, then broader structural rewrites if needed.

Produce at least two optimized alternatives, each with: - why it should improve the specific plan bottleneck - semantic risk checks to keep result parity - expected plan-level change signals to confirm improvement

Close with a local verification sequence: - equivalence checks (counts, key aggregates, sampled row parity) - side-by-side plan inspection points - repeated timing instructions and simple acceptance criteria